

MODELROUTER

Adaptive AI Inference & Model Routing Platform

Software Requirements Specification (SRS) + Full Development Blueprint

Purpose: A complete build specification for a Java/Spring Boot product that sits between customer applications and multiple AI model providers, choosing the best model per request while optimizing cost, latency, quality and reliability.

Item	Specification
Product	ModelRouter
Positioning	AI inference gateway / model-routing control plane
Primary goal	Optimize model selection without requiring application code to manage providers
Target users	AI startups, SaaS teams, enterprises, AI-agent builders
Backend	Java 21 + Spring Boot
Data	PostgreSQL + Redis
Messaging	Kafka — V2
Frontend	Next.js + TypeScript + Tailwind
Deployment	Docker → AWS

Core idea: The customer's application calls one stable ModelRouter API. ModelRouter evaluates the request, filters eligible models, scores them using policy and live signals, calls the selected provider, handles failures, records usage/cost/latency and returns the response.

1. Executive Summary

ModelRouter is an infrastructure product rather than a chatbot. A customer may use several AI models because models differ in quality, cost, latency, context size, capabilities and availability. ModelRouter centralizes that decision behind one API.

Simple mental model: ModelRouter answers “Which model should handle this request?” while the customer's application remains unaware of provider-specific routing logic.

```
Customer Application
|
v
ModelRouter API
|
+--> Security / Policy
+--> Request Classification
+--> Candidate Filtering
+--> Routing / Scoring
+--> Cache
|
+--> Provider A
+--> Provider B
+--> Provider C
|
+--> Usage / Cost / Metrics
|
v
Response
```

1.1 Product vision

Make AI model selection a reusable platform capability. A company integrates once and can later change providers, routing policies, budgets or fallback strategy without rewriting its application.

1.2 What it is not

- Not an LLM or model-training platform.
- Not a generic AI chatbot.
- Not a provider-specific SDK wrapper.
- Not a giant microservice architecture in the MVP.

2. Business Problem

Problem	Current situation	ModelRouter solution
Cost	Premium model used for easy requests	Route easy work to cheaper models
Reliability	Provider outage breaks application	Automatic fallback and health-aware routing
Performance	One model may be slow	Prefer low-latency models when policy allows
Provider lock-in	Application tightly coupled to one API	Provider adapter abstraction
Visibility	AI spend is hard to understand	Request-level usage/cost dashboard
Operations	Routing logic duplicated across teams	Central policy and routing layer

2.1 Example business case

```
100,000 AI requests/day

Before:
100% -> premium model

After:
60% -> low-cost model
30% -> balanced model
10% -> premium model
```

Track:
cost/request
latency
quality
fallback rate
cache hit rate

Business goal:
reduce cost while preserving acceptable quality.

2.2 Target customers

- AI startups with rapidly growing API bills.
- SaaS companies adding AI features.
- Enterprise engineering teams needing reliability and governance.
- AI-agent platforms that need different models for different tasks.
- Developer teams that want one API instead of multiple provider integrations.

3. Goals, Scope and Success Criteria

3.1 MVP goals

- One stable AI gateway API.
- At least three model choices through adapters.
- Explainable rule-based routing.
- Provider timeout and fallback.
- Redis caching.
- Usage, latency and cost tracking.
- Admin dashboard.
- Secure API-key authentication.
- Dockerized local setup.
- Automated unit/integration tests.

3.2 Explicitly out of MVP

- Training an AI routing model.
- Kubernetes.
- Complex multi-region deployment.
- Dozens of provider connectors.
- Full enterprise SSO.
- Autonomous agents.
- Production-grade semantic cache before exact cache works.

3.3 Future product scope

- Semantic cache and similarity search.
- Confidence-based model cascades.
- Circuit breaker and active health scoring.
- Per-team budgets and billing.
- A/B model experiments.
- Adaptive/learned routing.
- PII/prompt security gateway.
- Data residency policies.
- Agent-aware routing and governance.

3.4 Success metrics

Metric	Target / purpose
Routing overhead	<100 ms p95 excluding external model latency for MVP
Fallback success	Demonstrate recovery when primary provider is unavailable
Cache savings	Measure provider calls avoided
Cost optimization	Compare against single-premium-model baseline
Decision explainability	Every route should have a human-readable reason
Test coverage	Strong coverage of routing, failures and security paths

4. Functional Requirements

ID	Requirement
FR-01	Users can authenticate and create API keys.
FR-02	Admins can register, enable and disable providers/models.
FR-03	Admins can define routing policies and weights.
FR-04	Clients can send chat/inference requests to one gateway endpoint.
FR-05	System classifies task type and complexity.
FR-06	System filters ineligible/unhealthy models.
FR-07	System scores eligible models and selects the best candidate.
FR-08	System falls back after configured timeout/transient failure.
FR-09	System caches eligible repeated requests.
FR-10	System records tokens, latency, status, provider, model and estimated cost.
FR-11	Dashboard shows usage, cost, model distribution, failures and savings.
FR-12	Administrative changes generate audit records.

5. Non-Functional Requirements

Area	Requirement
Security	API keys hashed; provider secrets externalized; organization isolation enforced.
Performance	Stateless backend; low routing overhead.
Reliability	Provider failure should not automatically become application failure if a healthy fallback exists.
Scalability	Horizontal scaling should be possible without redesigning the core API.
Observability	Request ID, latency, status and routing trace for every request.
Maintainability	Provider-specific code isolated behind adapters.
Privacy	Prompt/response body logging configurable and redacted by policy.
Testability	Routing engine works with mock providers and no live API calls.

6. Actors and User Roles

Role	Permissions
Platform Admin	All providers, models, policies, budgets, audit and global analytics.
Organization Admin	Organization API keys, routing policies, budgets and analytics.
Developer/API User	Call inference API and view permitted usage.
Read-only Analyst	View analytics without changing configuration.

7. Main User Flows

7.1 Customer onboarding

- 1 Create organization.
- 2 Create API key.
- 3 Register models/providers.
- 4 Choose a routing mode.
- 5 Set a budget.
- 6 Send a test request.
- 7 Inspect routing decision and cost.
- 8 Simulate provider failure and verify fallback.

7.2 Request lifecycle

```
API request
-> API-key authentication
-> request validation
-> request/correlation ID
-> cache lookup
-> task/complexity classification
-> candidate model filtering
-> routing score calculation
-> primary provider call
-> success: record + cache + return
-> failure: fallback according to policy
-> final usage/cost/routing trace
```

8. Technical Architecture

```
+-----+
| Next.js Dashboard |
+-----+
|
v
Customer Apps -> Load Balancer -> Spring Boot API
|
+-----+
| | | |
v v v v v
Auth Classifier Router Redis Metrics
|
+-----+
| | |
v v v
Model A Model B Model C
| | |
+-----+
|
PostgreSQL

V2:
Spring Boot -> Kafka -> usage/analytics workers
```

8.1 Recommended Java package structure

```

com.modelrouter
  ■■■ auth
  ■■■ organization
  ■■■ user
  ■■■ api
  ■ ■■■ controller
  ■ ■■■ dto
  ■ ■■■ exception
  ■■■ routing
  ■ ■■■ classifier
  ■ ■■■ engine
  ■ ■■■ policy
  ■ ■■■ scoring
  ■■■ provider
  ■ ■■■ ModelProvider
  ■ ■■■ ProviderAAdapter
  ■ ■■■ ProviderBAdapter
  ■ ■■■ MockProviderAdapter
  ■■■ cache
  ■■■ usage
  ■■■ billing
  ■■■ health
  ■■■ audit
  ■■■ common

```

8.2 Design patterns

Pattern	Use
Strategy	Different routing/scoring policies.
Adapter	Normalize different provider APIs.
Factory	Create provider adapters.
Chain of Responsibility	Validation → security → budget → routing.
Circuit Breaker	Stop calls to unhealthy provider in V2.
Repository	Separate persistence from business logic.

9. Routing Engine — Core of the Product

The MVP should use deterministic and explainable routing. Machine-learning-based routing comes later. The first objective is to prove that a policy engine can make useful decisions and produce measurable savings/reliability improvements.

9.1 Model metadata

```
Model:
- provider
- model name
- enabled/disabled
- capabilities: chat, code, vision, long-context...
- context limit
- input price
- output price
- measured quality score
- measured latency
- reliability score
- current health
- priority
```

9.2 Candidate filtering

- Model enabled.
- Provider healthy.
- Capability matches request.
- Context window is sufficient.
- Organization policy allows it.
- Budget policy permits the expected request.

9.3 Weighted score

```
score =
qualityWeight * qualityScore
+ latencyWeight * latencyScore
+ costWeight * costScore
+ reliabilityWeight * reliabilityScore
+ capabilityWeight * capabilityMatch
```

Example BALANCED policy:

```
quality 0.35
latency 0.20
cost 0.20
reliability 0.20
capability 0.05
```

Weights must sum to 1.0.

9.4 Routing modes

Mode	Behavior
CHEAP	Strongly favor low cost.
FAST	Strongly favor low latency.
QUALITY	Strongly favor quality.
BALANCED	Default weighted policy.
CUSTOM	Organization-defined weights and constraints.

9.5 Explainable decision

```
Request:
"Design a distributed payment architecture."
```



```
Classification:
task = architecture
complexity = high
quality = high
```

```
Scores:
CheapModel 0.61
BalancedModel 0.78
ReasoningModel 0.91
```

```
Decision:
ReasoningModel
```

```
Reason:
High complexity + high quality requirement.
```

9.6 Confidence escalation — V2

```
Request
|
Cheap model
|
confidence >= threshold?
/ YES NO
| |
return premium model
|
return
```

9.7 Adaptive routing — V3

Collect real outcomes by task category. Compare model quality, latency and cost. Adjust routing policies based on measured performance rather than hard-coded assumptions. A learned router can eventually replace or augment the weighted policy.

10. Database Design

Use PostgreSQL as the source of truth. Keep the schema relational and easy to understand.

Table	Key fields	Purpose
users	id, email, password_hash, role, status	Application users.
organizations	id, name, plan, budget_limit	Tenant boundary.
api_keys	id, organization_id, key_hash, status, last_used_at	Programmatic access.
providers	id, name, base_url, status	Provider registry.
models	id, provider_id, name, capabilities, prices, status	Model catalog.
routing_policies	id, organization_id, mode, weights_json	Routing rules.
routing_requests	id, org_id, request_id, selected_model_id, status, latency_ms, input_tokens, output_tokens, cost	Usage.
routing_attempts	id, request_id, model_id, attempt_no, status, latency_ms, error	Fallback trace.
audit_logs	id, actor_id, action, resource, created_at	Admin audit trail.

10.1 Important indexes

- users(email) unique.
- api_keys(key_hash) unique.
- routing_requests(organization_id, created_at).
- routing_requests(request_id) unique.
- routing_attempts(request_id, attempt_no).
- models(provider_id, status).

11. API Specification

11.1 Main inference endpoint

```
POST /api/v1/chat
Authorization: Bearer

{
  "messages": [
    {"role": "user", "content": "Explain Java interfaces"}
  ],
  "mode": "balanced",
  "maxTokens": 500
}

Response:
{
  "requestId": "req_123",
  "model": "model-b",
  "provider": "provider-b",
  "content": "...",
  "usage": {
    "inputTokens": 120,
    "outputTokens": 260,
    "estimatedCost": 0.08
  },
  "routing": {
    "mode": "balanced",
    "reason": "Medium complexity; cost/quality balance"
  }
}
```

11.2 Admin endpoints

Method	Endpoint	Purpose
POST	/api/v1/admin/providers	Register provider.
GET	/api/v1/admin/providers	List providers.
POST	/api/v1/admin/models	Register model.
PATCH	/api/v1/admin/models/{id}	Enable/disable model.
POST	/api/v1/admin/policies	Create routing policy.
GET	/api/v1/admin/analytics/overview	Dashboard summary.
GET	/api/v1/admin/requests	Request explorer.
GET	/api/v1/admin/health/models	Model health.

11.3 API rules

- Use DTOs; never expose JPA entities directly.
- Return requestId in errors.
- Validate token/request limits.
- Never expose provider credentials.
- Enforce organization isolation.
- Support configurable body logging/redaction.

12. Redis Design

Use	Key example	Purpose
Exact response cache	cache:chat:<hash>	Avoid duplicate provider calls.
Model health	health:model:<id>	Fast health state.
Rate limit	rate:<org>:<window>	Request control.
Routing policy cache	routing:policy:<org>	Reduce DB reads.

12.1 Semantic cache — V2

Convert request text to an embedding, search for sufficiently similar previous requests, and return a cached answer only when policy allows. Do not start here; exact-match caching is enough for the MVP.

13. Security

- Spring Security for authentication/authorization.
- Hash API keys; show secret only once at creation.
- Hash passwords with an adaptive password hashing algorithm.
- Provider keys only through environment/secret management.
- RBAC for admin endpoints.
- Strict organization/tenant isolation.
- Rate limits per organization/API key.
- Configurable prompt/response logging and PII redaction.
- HTTPS in production.
- Audit provider/model/policy/key/budget changes.

13.1 Security flow

```
API Key
-> authenticate
-> resolve organization
-> authorize role
-> rate/budget check
-> validate request
-> route
-> provider
```

14. Reliability

Mechanism	MVP	V2
Timeout	Yes	Configurable per provider
Fallback	Yes	Multi-provider policy
Retry	Limited transient retry	Exponential backoff
Circuit breaker	Optional	Yes
Health	Basic status	Active + passive scoring
DLQ	No	Kafka DLQ for event failures

14.1 Failure example

```
Primary provider
|
timeout
|
```

```
fallback provider
|
success -> return + record attempt history
```

```
If all providers fail:
return stable ModelRouter error
with requestId and safe message.
```

15. Observability

- Request ID on every request.
- Model selection and reason.
- Provider latency and status.
- Token usage and estimated cost.
- Fallback count.
- Cache hit/miss.
- Error rate.
- Routing overhead p50/p95/p99.

16. Dashboard Requirements

Screen	Contents
Overview	Requests, cost, estimated savings, latency, errors, cache hit rate.
Models	Provider/model list, health, price, capabilities, enable/disable.
Routing Policies	Modes, weights, allowed models, budget.
Request Explorer	Request ID, selected model, attempts, latency, cost, status.
Model Comparison	Quality, latency, cost, reliability.
API Keys	Create/revoke keys; never display secret again.
Audit	Administrative actions.

17. Frontend Technology

Next.js + TypeScript + Tailwind + shadcn/ui. For vibe-coding, use the frontend mainly as an operational dashboard. Do not spend weeks building a complex marketing site before the backend works.

18. Testing Strategy

Test	Examples
Unit	Scoring, normalization, classification, candidate filtering, budget rules.
Integration	Spring Boot + PostgreSQL + Redis.
Provider contract	Adapter request/response mapping.
Failure	Timeout, 500, malformed response, fallback.
Security	Unauthorized key, wrong organization, role violations.
Concurrency	Concurrent rate/budget updates.
E2E	Client → gateway → mock provider → response + metrics.
Load	Increasing concurrent requests and routing overhead.

18.1 Recommended tools

- JUnit 5
- Mockito
- Spring Boot Test
- Testcontainers
- WireMock or MockWebServer
- Postman/Newman

19. Docker / Local Environment

```
docker compose up
```

MVP:

- modelrouter-api
- postgres
- redis
- frontend

V2:

- kafka
- prometheus
- grafana

20. AWS Deployment

```
Internet
|
HTTPS Load Balancer
|
Spring Boot containers (stateless)
|
+-----+
| |
RDS PostgreSQL ElastiCache Redis

CI/CD:
GitHub
-> tests
-> Docker build
-> image registry
-> staging
-> smoke test
-> production
```

21. Development Roadmap

Build vertically: every phase must leave a working application. Do not start with Kafka, Kubernetes or learned routing.

Phase	Development work	Output
0	SRS, architecture, repo, env config	Approved blueprint
1	Java 21, Spring Boot, PostgreSQL, health endpoint	Running backend
2	Users, organizations, API keys, security	Secured gateway
3	ModelProvider interface + mock + real adapters	Provider abstraction
4	Classifier + candidate filtering + scoring	Smart routing
5	Timeout + fallback + health	Resilience
6	Usage/cost/latency persistence	Analytics data
7	Redis cache + rate limit	Performance/cost controls
8	Next.js dashboard	Product UI
9	Tests + Testcontainers + failure tests	Quality gate
10	Docker + README + demo scripts	Portable product
11	AWS + monitoring	Public demo
12	Semantic cache + Kafka + adaptive routing	Advanced product

22. Six-Week Practical Schedule

Week	Target
1	Foundation, DB, authentication, API keys.
2	Provider abstraction, mock provider, 2 real provider adapters, core endpoint.
3	Classifier, routing score, routing modes, explainability, fallback.
4	Redis cache, usage/cost tracking, health and admin APIs.
5	Dashboard, tests, Docker, polished README.
6	AWS deployment, observability, benchmark, demo video and resume bullets.

23. Vibe-Coding Development Rules

- Ask AI to implement bounded modules, not the entire product in one prompt.
- Write the API contract and schema before generating code.
- Use AI for DTOs, controllers, repetitive CRUD, tests and UI boilerplate.
- Manually inspect routing, security, transaction, caching and failure logic.
- Run tests after every meaningful change.
- Keep commits small and meaningful.
- Create ADRs for major architecture decisions.
- Do not add a technology unless you can explain why it exists.

24. Suggested GitHub Structure

```
modelrouter/  
■■■ backend/  
■■■ frontend/  
■■■ docs/  
■ ■■■ SRS.md
```



```
■ ■■■ architecture.md
■ ■■■ api.md
■ ■■■ ADR/
■■■ docker-compose.yml
■■■ .github/workflows/
■■■ README.md
■■■ LICENSE
```

25. MVP Acceptance Criteria

- Developer can create API key and call one inference endpoint.
- At least three model choices are available through adapters.
- Routing decision is deterministic and explainable.
- Primary provider timeout causes configured fallback.
- Eligible repeated requests can be served from Redis.
- Usage, latency, status and estimated cost are stored.
- Dashboard displays model distribution, cost and failures.
- Provider can be disabled without application-code changes.
- Tests cover routing and provider failures.
- Entire stack starts with Docker Compose.
- README contains architecture, setup, API examples and design decisions.

26. Benchmark Plan

Experiment	Measure
Premium-only baseline	Cost/request, latency, quality.
Rule-based routing	Cost/request, latency, quality.
Cache enabled	Hit rate, calls avoided, cost reduction.
Provider failure	Recovery/success rate and added latency.
Concurrency	Requests/sec, error rate, routing overhead.

27. Demo Script

1. Send easy Java question -> cheap model selected.
2. Send complex architecture question -> stronger model selected.
3. Disable primary provider -> fallback succeeds.
4. Repeat request -> cache hit.
5. Open dashboard -> cost, latency, model distribution.
6. Change routing weights -> model selection changes.
7. Show request trace -> exact reason for selection.

28. Product Monetization

The product can start as a developer SaaS and later move toward enterprise infrastructure.

Plan	Example offering
Free	Limited monthly requests, basic routing and dashboard.
Developer	Higher limits, caching, fallback, analytics.
Startup	Budgets, advanced policies, team usage, more providers.
Enterprise	SSO, private deployment, governance, audit, data residency.

29. Business Value Proposition

Pitch: “Connect your AI application once. ModelRouter automatically chooses the best model for every request, fails over when providers fail, and gives your team visibility and control over AI cost and performance.”

The product's strongest measurable KPI is cost per successful task, not merely requests routed. A good benchmark should demonstrate that routing reduces cost or latency without unacceptable quality loss.

30. Future Product Roadmap

MVP

- > multi-provider gateway
- > rule-based routing
- > cost/latency
- > fallback
- > cache

V2

- > semantic cache
- > confidence cascade
- > budgets
- > A/B tests
- > Kafka
- > model health

V3

- > adaptive/learned routing
- > security/PII gateway
- > governance
- > data residency
- > agent-aware routing

Platform

- > enterprise SSO
- > billing
- > private deployment
- > provider marketplace

31. Final Technology Stack

Layer	Technology	Purpose
Language	Java 21	Primary backend language.
Framework	Spring Boot	REST, DI, production backend.
Security	Spring Security + API keys/JWT	Auth and RBAC.
ORM	Spring Data JPA + Hibernate	Persistence.
Database	PostgreSQL	Users, models, policies, usage.
Cache	Redis	Cache, rate limits, health metadata.
Messaging	Kafka (V2)	Async usage/events.
Frontend	Next.js + TypeScript + Tailwind	Dashboard.
Testing	JUnit 5, Mockito, Testcontainers	Automated tests.
Observability	Actuator, Prometheus, Grafana	Metrics and monitoring.
DevOps	Docker, GitHub Actions	CI/CD.
Cloud	AWS	Deployment.

32. Interview Preparation From This Project

- Why Java/Spring Boot?
- Why PostgreSQL?
- Why Redis?
- How does the routing score work?
- How are cost and latency normalized?
- How does fallback differ from retry?
- How would you implement circuit breaking?
- How do you secure provider keys?
- How do you prevent cross-tenant data access?
- How would you scale to 10,000 requests/second?
- What should move to Kafka and why?
- How would you add a new provider without changing routing logic?
- How would you prove that routing actually saves money?

33. Resume Positioning

ModelRouter — Adaptive AI Inference & Model Routing Platform

- Built a Spring Boot AI gateway that routes inference requests across multiple model providers using task complexity, cost, latency and reliability policies.
- Implemented provider-agnostic adapters, automatic fallback and Redis-based caching to improve resilience and reduce unnecessary model calls.
- Designed request-level telemetry for model selection, token usage, latency and estimated inference cost with an operational analytics dashboard.

Only use measured numbers after running the benchmark. Never invent savings, performance or scale figures.

34. Final Build Principle

Build the smallest complete system first: one API, multiple providers, deterministic routing, fallback, cache, usage tracking and dashboard. Then measure it. Only after the baseline works should you add Kafka, semantic caching, adaptive routing or advanced AI governance.

Final mental model: The customer application should not need to care which AI model is used. ModelRouter owns that decision and makes it measurable, explainable, reliable and optimizable.

END OF MODELROUTER SRS — VERSION 1.0